Resumen esencial para el examen de PSP

PARTE 1 · PROCESOS (presentación 02)

1.1 Conceptos que debes saber distinguir

Concepto	Qué es
Programa / código	Entidad **pasiva**: el código almacenado (`MiAplicacion.kt`, `mi-app.jar`).
Proceso	**Instancia concreta en ejecución** de un programa: tiene su propio estado, memoria, recursos e hilos.
Hilo	Flujo de ejecución independiente **dentro de un proceso**. Los hilos de un proceso **comparten** los recursos de ese proceso.
Servicio	Funcionalidad o componente gestionado que presta una función continuada o bajo demanda. Puede apoyarse en uno o más procesos.

Servicio ≠ proceso ≠ hilo.

- Un mismo programa puede tener **varias ejecuciones independientes**: tres procesos de `mi-app.jar` tienen cada uno su memoria y su `contador`. Una variable de un proceso **no es** la misma variable de otro.
- Procesos distintos tienen **mucho mayor aislamiento** que hilos del mismo proceso.
- Terminología: **proceso hijo ≠ hilo**. Un proceso hijo es un proceso creado por otro proceso.

1.2 Concurrencia, paralelismo y distribución

- **Concurrencia**: varias tareas **progresan durante un mismo intervalo de tiempo**, aunque no se ejecuten a la vez. Se puede dar con un solo núcleo.
- **Paralelismo**: dos o más tareas se ejecutan **realmente al mismo tiempo**.
- **Distribución**: componentes que se ejecutan en **máquinas diferentes** conectadas por red. Costes: latencia, fallos parciales, complejidad.
- **No son alternativas excluyentes**: un sistema distribuido puede ser además concurrente y paralelo.
- **La concurrencia no requiere paralelismo.**

1.3 ¿Cuándo merece la pena la concurrencia?

- **Sí**: tareas independientes con muchas esperas de E/S (se pueden solapar).

- **No**: B necesita inmediatamente el resultado completo de A, o las tareas son muy pequeñas (coordinar cuesta más que el trabajo).
- **Depende**: un cálculo grande divisible en partes independientes (tamaño, recursos, coste de coordinación).
- Regla: **más concurrencia ≠ programa mejor**. Añade coordinación, complejidad y nuevos errores posibles.

Preguntas útiles: ¿son independientes?, ¿hay esperas?, ¿necesito capacidad de respuesta?, ¿qué coordinación añado?, ¿qué errores nuevos pueden aparecer?

Qué usar en cada caso (pregunta típica de examen):

- Dos herramientas externas independientes → **procesos**.
- Misma aplicación, tareas que comparten objetos en memoria → **hilos**.
- Cálculo con dependencia inmediata, o trabajos diminutos → **secuencial**.

1.4 Estados de un proceso (modelo simplificado)

`NUEVO → LISTO → EN EJECUCIÓN → TERMINADO`, y desde ejecución puede pasar a `BLOQUEADO/EN ESPERA` (espera de evento o E/S) y de ahí vuelve a `LISTO`.

Transiciones esenciales:

- listo → ejecución (obtiene CPU)
- ejecución → listo (deja la CPU)
- ejecución → bloqueado (espera E/S)
- bloqueado → listo (el evento ocurre)
- ejecución → terminado

Idea clave: mientras un proceso A espera E/S, el SO puede ejecutar B. **No hacen falta dos núcleos para que haya concurrencia.**

1.5 ProcessBuilder: lanzar un proceso hijo

Kotlin

```
val process = ProcessBuilder("java", "--version").start()
val exitCode = process.waitFor()
val output = process.inputStream.bufferedReader().use {
it.readText() }
println(output.trim())
println("Código de salida: $exitCode")
```

- `ProcessBuilder("java", "--version")`: comando + argumentos (cada uno es un String aparte).
- `start()`: **inicia** el proceso y devuelve un objeto `Process`.

- `waitFor()`: **espera** a que termine y devuelve el **código de salida** (0 = normalmente éxito).
- `isAlive`: consulta **en ese instante** si aún no ha terminado. **No espera.** En un proceso muy breve puede dar ya `false`.

Lanzar ≠ observar ≠ esperar (`start()` / `isAlive` / `waitFor()`).

1.6 Comunicación con el hijo: stdin, stdout, stderr

Quiero…	Uso
Escribir al **stdin** del hijo	`process.outputStream`
Leer el **stdout** del hijo	`process.inputStream`
Leer el **stderr** del hijo	`process.errorStream`

Los nombres parecen al revés, pero se ven **desde el padre**: lo que el padre *escribe* sale (output) hacia el hijo; lo que el padre *lee* entra (input) desde el hijo.

Comunicación ≠ espera: los streams transportan datos; `waitFor()` espera la terminación.

Ejemplo completo (hijo en `ChildWorker.jar` que lee una línea y la devuelve en mayúsculas):

```kotlin
val process = ProcessBuilder("java", "-jar", "ChildWorker.jar", "A").start()

process.outputStream.bufferedWriter().use { it.write("hola\n") }
// escribo al stdin del hijo (con \n)
val code = process.waitFor()
val out = process.inputStream.bufferedReader().use { it.readLine() }
val err = process.errorStream.bufferedReader().use { it.readLine() }
```

Salida: `stdout -> A -> HOLA`, `stderr -> A: fin`, `código -> 0`.

Detalles importantes:

- El `\n` al escribir es necesario porque el hijo usa `readln()`, que espera una línea completa.
- El `.use { }` cierra el stream al terminar. Cerrar el stdin avisa al hijo de que no hay más entrada.
- Aviso de la presentación: **esperar antes de leer no es una receta general**. Aquí es seguro porque el hijo produce solo dos líneas. Con mucha salida, el hijo podría bloquearse.

1.7 El error clave: serialización accidental

```kotlin
// ❌ Elimina la concurrencia
val a = launchChild("A")
a.waitFor()
val b = launchChild("B")   // B no existe hasta que A termina
b.waitFor()

// ✓ Lanzar primero, esperar después
val a = launchChild("A")
val b = launchChild("B")
a.waitFor()
b.waitFor()
```

Patrón correcto: **start A, start B, → wait A, wait B**. Así ambos pueden existir y progresar en el mismo intervalo.

1.8 Qué se puede y qué no se puede afirmar

Tras lanzar A y B antes de esperar:

- ✓ Coexistieron como procesos (1 padre + 2 hijos).
- ✓ Pudieron progresar **concurrentemente**.
- ✗ **No** se deduce **paralelismo físico**.
- ✗ Que la salida de A aparezca primero **no demuestra** que A terminara antes: solo es que el padre imprime A primero.
- Después de `waitFor()`: `process.isAlive == false`.

PARTE 2 · HILOS SIN DATOS COMPARTIDOS (presentación 03)

2.1 Idea base

Un proceso JVM ya tiene un hilo desde el principio: **`main`**. Puedes crear más. Todos comparten la memoria del proceso, pero en esta parte cada hilo trabaja con sus propios datos locales.

`Thread.currentThread().name` te dice **qué hilo ejecuta esa línea**.

```kotlin
fun main() {
    println("main -> ${Thread.currentThread().name}")   // main
    val worker = Thread { println("trabajo ->
${Thread.currentThread().name}") }
    worker.name = "worker-A"
    worker.start()                                       //
trabajo -> worker-A
}
```

2.2 Dos formas de crear un hilo (debes saber leer y escribir ambas)

```kotlin
// 1. Subclase de Thread
class MiHilo : Thread() {
    override fun run() { println("Trabajo") }
}
val hiloA = MiHilo()

// 2. Thread con lambda
val hiloB = Thread { println("Trabajo") }
```

- `run()` contiene **el trabajo** que ejecutará el hilo.
- `start()` **arranca** el hilo.

2.3 `run()` vs `start()` (el error más importante)

- `worker.run()` → **llamada normal**: lo ejecuta el hilo que llama (por ejemplo `main`). **No se crea ningún hilo nuevo.**
- `worker.start()` → **arranca el hilo** `worker`, que ejecutará `run()` en su propio flujo.

```kotlin
worker.run()    // imprime "Trabajo en main"
worker.start()  // imprime "Trabajo en worker-A"
```

2.4 Varios hilos y el orden

```kotlin
workerA.start()
workerB.start()
```

- Dentro de cada hilo el orden se respeta: `A0 → A1 → A2 → A3`.
- **Entre A y B no hay orden garantizado.**
- **Una ejecución observada no crea una garantía.**

2.5 `sleep()` vs `join()`

	`Thread.sleep(ms)`	`worker.join()`
Responde a	¿esperar **cuánto tiempo**?	¿ha **terminado** otro hilo?
Quién espera	El hilo que **ejecuta** esa llamada	El hilo que **ejecuta** `join()`
Estado mientras espera	`TIMED_WAITING`	`WAITING`

- `join()`: el hilo que lo llama no pasa de esa línea hasta que `worker` termine. Si ya había terminado, retorna inmediatamente.
- `sleep()` puede cortarse antes si hay una **interrupción**.
- **Puente con procesos**: `Process.waitFor()` = esperar terminación de un proceso; `Thread.join()` = esperar terminación de un hilo.

2.6 `Thread.State` (¡no son los estados de proceso!)

Estado	Significado
`NEW`	Creado, todavía no iniciado
`RUNNABLE`	Ejecutándose en la JVM (puede estar esperando CPU del SO)
`BLOCKED`	Esperando adquirir un **monitor**
`WAITING`	Esperando indefinidamente una acción
`TIMED_WAITING`	Esperando un tiempo determinado
`TERMINATED`	Ha finalizado

`Thread.State` NO tiene `RUNNING`. Ciclo típico:

Kotlin

```kotlin
val worker = Thread { Thread.sleep(2000) }
println(worker.state)   // NEW
worker.start()
Thread.sleep(100)
println(worker.state)   // normalmente TIMED_WAITING
worker.join()
println(worker.state)   // TERMINATED
```

"Bloqueado" como idea general ≠ `Thread.State.BLOCKED`. Durante un `sleep` el estado es `TIMED_WAITING`, no `BLOCKED`.

2.7 `interrupt()`: cancelación cooperativa

`interrupt()` no mata el hilo: solicita que se detenga. El hilo debe cooperar:

- Si está ejecutando código, puede consultar `isInterrupted`.
- Si está en una operación interrumpible (`sleep`, `join`...), lanza **`InterruptedException`**.

Kotlin

```kotlin
class Worker : Thread("worker") {
    override fun run() {
        try {
            while (!isInterrupted) {
                println("trabajando")
                Thread.sleep(500)
            }
        } catch (e: InterruptedException) {
            println("interrumpido durante sleep")
        }
        println("fin")
    }
}

val worker = Worker()
worker.start()
Thread.sleep(1200)
worker.interrupt()   // solicita parar
worker.join()        // confirma que terminó
```

Los métodos `Thread.stop()`, `suspend()` y `resume()` **no se usan** para detener o pausar.

2.8 Un `Thread` solo se arranca una vez

- Tras terminar, el objeto queda en `TERMINATED`. Hacer `start()` otra vez lanza `IllegalThreadStateException` (no hace falta memorizar el nombre).
- No existe `TERMINATED → NEW` en el mismo objeto.
- **Reiniciar la tarea = crear un nuevo `Thread`**, no arrancar el mismo otra vez.

2.9 Prioridad

Se puede consultar (`priority`) y modificar dentro de su rango, pero **no garantiza** qué hilo se ejecuta o termina primero, ni un intercalado concreto, ni la corrección. **Nunca la uses para coordinar.**

2.10 El ejercicio del Segundero (muy probable en examen)

Kotlin

```kotlin
class Segundero : Thread("segundero") {
    override fun run() {
        var segundos = 0                // dato LOCAL a esta ejecución
        try {
            while (true) {
                println("$segundos · " + Thread.currentThread().name)
                Thread.sleep(1000)      // espera ~1 s
                segundos++              // suma 1
            }
        } catch (e: InterruptedException) {
            println("Segundero detenido")   // fin de run() → TERMINATED
        }
    }
}

fun main() {
    var segundero = Segundero()
    segundero.start()
    Thread.sleep(3500)          // main solo deja pasar tiempo
    segundero.interrupt()       // solicita parada
```

```
    segundero.join()              // espera finalización
    println("1: vivo=${segundero.isAlive},
estado=${segundero.state}")  // false, TERMINATED

    segundero = Segundero()       // OTRO objeto
    segundero.start()
    Thread.sleep(2500)
    segundero.interrupt()
    segundero.join()
    println("2: vivo=${segundero.isAlive},
estado=${segundero.state}")
}
```

Puntos a explicar:

- El 0 sale inmediatamente, porque el `println` va antes del `sleep`.
- `segundos` es local: no hay estado compartido. No se usan variables `running` compartidas ni `synchronized`.
- `Thread.sleep(3500)` en `main` solo deja transcurrir tiempo; **la finalización se comprueba con `join()`**.
- La segunda ejecución vuelve a empezar en 0 porque es un objeto nuevo.

2.11 Cómo explicar un programa multihilo

1. ¿Qué hilos existen? 2. ¿Qué hilo ejecuta cada parte? 3. ¿Qué hace cada operación (`start` arranca, `sleep` espera tiempo, `join` espera finalización, `interrupt` solicita parar)? 4. ¿Ha terminado (`isAlive`, `state`)? 5. ¿Qué orden está garantizado? **El orden establecido por el programa ≠ el orden observado una vez.**

PARTE 3 · HILOS CON DATOS COMPARTIDOS (presentación 04)

3.1 Estado mutable compartido

- Variable **local** (`var segundos` dentro de `run()`): cada hilo tiene la suya.
- Objeto **compartido** (`val cuenta = Cuenta()` usado por dos hilos): ambos acceden al mismo objeto y a su estado, porque comparten la referencia.
- **Compartir un dato ≠ compartirlo correctamente.**

3.2 El ejemplo de la cuenta

```kotlin
class Cuenta {
    var saldo = 0
    fun ingresar(cantidad: Int) { saldo += cantidad }
}
```

- `mete-10` hace 1 000 000 ingresos de 10 y `mete-5` hace 1 000 000 de 5, sobre la **misma** cuenta.
- **Invariante**: si no se pierde ningún ingreso, el saldo final debe ser 1 000 000·10 + 1 000 000·5 = **15 000 000**, sea cual sea el orden.
- Sin protección, el resultado observado suele ser **menor**.
- Estructura del `main`: `start` de ambos → `join` de ambos → comprobar saldo. Se arrancan los dos **antes** de esperar.
- **Una ejecución correcta no demuestra que el programa sea correcto**: hay que razonar sobre un interleaving posible.

3.3 Por qué falla: `saldo += cantidad` no es una sola operación

Se modela en tres pasos:

1. **LEER** `actual = saldo`
2. **CALCULAR** `nuevo = actual + cantidad`
3. **ESCRIBIR** `saldo = nuevo`

Lo mismo ocurre con `contador++`.

Atomicidad: una operación es atómica cuando se comporta como una unidad indivisible para la coordinación que analizamos. `saldo += cantidad` **no debe asumirse atómica**.

3.4 Actualización perdida (saldo inicial 0)

A (+10)	B (+5)	saldo
lee 0		0
	lee 0	0
calcula 10		0
	calcula 5	0
escribe 10		10

escribe **5**
5

Esperado 15, obtenido **5**. La actualización de A se sobrescribe.

Claves:

- Ningún cálculo individual estaba mal; el fallo viene del **interleaving** sobre el mismo estado.
- **No requiere paralelismo físico**: basta con que los pasos se intercalen.

3.5 Condición de carrera

La corrección del resultado depende de un orden o interleaving **no controlado** de operaciones concurrentes que acceden a **estado compartido**.

Ingredientes: concurrencia + estado mutable compartido + accesos que interfieren + sin coordinación adecuada.

Orden variable ≠ error automáticamente. Que A imprima antes que B, o al revés, puede ser correcto (variabilidad permitida). Es carrera **problemática** solo si algún interleaving **viola la invariante**.

3.6 Atomicidad y visibilidad

- **Atomicidad**: ¿puede otro hilo colarse dentro de una operación que debe ser una unidad? LEER → CALCULAR → ESCRIBIR no debe quedar expuesto.
- **Visibilidad**: cuando los hilos se coordinan, necesitamos garantías sobre qué cambios de un hilo pueden ver los demás.
- Nivel exigido: comprender ambas necesidades y usar el **mismo monitor** correctamente. **No** entra: Java Memory Model formal, cachés/RAM, `volatile`, APIs atómicas.

3.7 Sección crítica y exclusión mutua

- **Sección crítica**: parte del programa que accede a estado compartido y cuya ejecución concurrente no controlada puede romper la corrección. Aquí: `saldo += cantidad` completo (leer-calcular-escribir).
- Debe protegerse **la operación crítica completa**, pero no todo `run()`.
- **Exclusión mutua**: respecto del **mismo mecanismo de bloqueo**, como máximo un hilo ejecuta la sección protegida a la vez. A entra, B espera, A sale, B entra.
- Exclusión mutua ≠ eliminar toda la concurrencia: solo se restringe lo que necesita protección. El objetivo es la **corrección**.

3.8 `synchronized(lock)`

```kotlin
class Cuenta {
    var saldo = 0
        private set
    private val lock = Any()

    fun ingresar(cantidad: Int) {
        synchronized(lock) {
            saldo += cantidad
        }
    }
}
```

- `private val lock = Any()`: objeto **estable** (uno por `Cuenta`) cuyo **monitor** se usa para coordinar.
- Para entrar al bloque, el hilo debe **adquirir el monitor** de `lock`.
- **Mismo objeto lock → mismo monitor.**
- `private set` impide que desde fuera se modifique `saldo` saltándose la protección.

3.9 Error típico: lock distinto en cada llamada

```kotlin
// ❌ Incorrecto
fun ingresar(cantidad: Int) {
    val lock = Any()            // nuevo lock en CADA llamada
    synchronized(lock) { saldo += cantidad }
}
```

Cada llamada compite por un monitor distinto, así que A y B **pueden entrar a la vez**. La exclusión mutua solo existe si **compiten por el mismo monitor**. Además, **todos** los accesos al estado compartido deben seguir el mismo protocolo con el mismo lock.

3.10 Con `synchronized` el orden varía, el saldo no

- Se mantiene el `main` igual (start, start, join, join, comprobar).
- No se garantiza un orden A → B → A → B, **pero sí** que el saldo final sea **15 000 000**.
- Si B intenta adquirir un monitor que tiene A, B puede estar en `Thread.State.BLOCKED` (conexión con la parte 2).

3.11 `join()` y `synchronized` resuelven problemas distintos

	`join()`	`synchronized(lock)`
Pregunta	¿Cuándo puede continuar el hilo que espera?	¿Quién puede entrar ahora en la sección protegida?
Garantiza	Que el worker **ha terminado** antes de comprobar	Que las actualizaciones **no se intercalen**
Excluye a A y B entre sí	**No**	**Sí**

Resumen: `synchronized` → actualizaciones correctas; `join` → ambos terminados antes de comprobar el resultado. **`join()` no sustituye a `synchronized`.**

3.12 Las cinco preguntas para revisar código multihilo

1. ¿Qué estado comparten?
2. ¿Qué accesos pueden interferirse?
3. ¿Cuál es la sección crítica?
4. ¿Todos esos accesos siguen el mismo protocolo de protección?
5. ¿Cómo compruebo y documento la corrección?

Una ejecución correcta no basta: hay que poder explicar por qué el programa es correcto.

Chuleta final (lo que más probablemente te pregunten)

- **Proceso vs hilo**: el proceso tiene memoria propia y aislada; los hilos de un proceso comparten memoria.
- **Concurrencia vs paralelismo**: concurrencia = progresan en el mismo intervalo; paralelismo = ejecución real simultánea. La primera no exige la segunda.
- **Procesos**: `ProcessBuilder(...).start()`, `waitFor()` (código de salida), `isAlive` (no espera), streams `outputStream` (escribo), `inputStream` y `errorStream` (leo).
- **Lanzar todos antes de esperar** para no perder concurrencia.
- **Hilos**: `start()` crea el hilo, `run()` directo no; `sleep` = tiempo, `join` = terminación; `interrupt` solicita, no mata; un `Thread` solo se arranca una vez; no hay estado `RUNNING`.
- **Datos compartidos**: `+=` son tres pasos → actualización perdida → condición de carrera → sección crítica → `synchronized` con el **mismo lock** → invariante preservada.
- **Frase de oro**: orden observado ≠ orden garantizado, y una ejecución correcta no demuestra que el programa sea correcto.

Si quieres, te preparo unas preguntas tipo test o ejercicios de código para que practiques antes del examen.